Шаблонизация проектов DS

MLOps

Cookiecutter DS

```
├── LICENSE            <- Open-source license if one is chosen
├── Makefile           <- Makefile with convenience commands like `make data` or `make train`
├── README.md          <- The top-level README for developers using this project.
├── data
│   ├── external       <- Data from third party sources.
│   ├── interim        <- Intermediate data that has been transformed.
│   ├── processed      <- The final, canonical data sets for modeling.
│   └── raw            <- The original, immutable data dump.
│
├── docs               <- A default mkdocs project; see www.mkdocs.org for details
│
├── models             <- Trained and serialized models, model predictions, or model summaries
│
├── notebooks          <- Jupyter notebooks. Naming convention is a number (for ordering),
│                         the creator's initials, and a short `-` delimited description, e.g.
│                         `1.0-jqp-initial-data-exploration`.
│
├── pyproject.toml     <- Project configuration file with package metadata for
│                         {{ cookiecutter.module_name }} and configuration for tools like black
│
├── references         <- Data dictionaries, manuals, and all other explanatory materials.
│
├── reports            <- Generated analysis as HTML, PDF, LaTeX, etc.
│   └── figures        <- Generated graphics and figures to be used in reporting
│
├── requirements.txt   <- The requirements file for reproducing the analysis environment, e.g.
│                         generated with `pip freeze > requirements.txt`
│
├── setup.cfg          <- Configuration file for flake8
│
└── {{ cookiecutter.module_name }}   <- Source code for use in this project.
    │
    ├── __init__.py             <- Makes {{ cookiecutter.module_name }} a Python module
    │
    ├── config.py               <- Store useful variables and configuration
    │
    ├── dataset.py              <- Scripts to download or generate data
    │
    ├── features.py             <- Code to create features for modeling
    │
    ├── modeling
    │   ├── __init__.py
    │   ├── predict.py          <- Code to run model inference with trained models
    │   └── train.py            <- Code to train models
    │
    └── plots.py                <- Code to create visualizations
```

Create Cookiecutter Template

- python -m venv ./venv
- source venv/bin/activate (для macOS)
- pip install cookiecutter
- cookiecutter https://github.com/drivendataorg/cookiecutter-data-science -c v1

Code Style

Форма социального контракта

- Что и почему важно?
- Стандарт PEP8
- Dockstring
- Аннотации типов
- Автоформатеры
- Линтеры
- CI/CD

Почему важно Code Style

- Чтение кода - самая большая часть времени в разработке
- Паттерны DRY, KISS, SOLID
- Увеличение кода
- Снижение эффективности команды из-за плохого кода
- Репутация компании

Из чего состоит Code Style

- Название переменных, функций, др.
- Комментарии
- Аннотации
- Dockstring
- Отступы, переносы
- Шаблоны проектирования

PEP 8

Python Enhancement Proposal 8

```
def example1():
    # This is a long comment. This should be wrapped to fit within 72
    # characters.
    some_tuple = (1, 2, 3, 'a')
    some_variable = {
        'long': 'Long code lines should be wrapped within 79 characters.',
        'other': [
            math.pi,
            100,
            200,
            300,
            9876543210,
            'This is a long string that goes on'],
        'more': {
            'inner': 'This whole logical line should be wrapped.',
            some_tuple: [
                1,
                20,
                300,
                40000,
                500000000,
                60000000000000000]}}
    return (some_tuple, some_variable)
```

- 4 пробела на каждый уровень отступа
- Висячие отступы, аргументы на первой линии запрещены

```
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

- 79 символов для кода (PEP 8 Default)
- 72 символов для документации и комментариев (PEP 8 Default)
- 88 символов (Black Default)
- 1 строка на метод внутри класса
- 2 строки на функции верхнего уровня и определения классов

PEP 8

Python Enhancement Proposal 8

- Каждый импорт на отдельной строчке
- Возможно подключение атрибутов одного модуля на одной строке
- Импорты всегда в начале файла
- Группировка осуществляется через пустую строку (стандартные, сторонние библиотеки, модули)
- Комментарий объясняющий код сверху на кодом
- Каждая строчка комментария начинается с # и один пробел после
- Абзацы комментарий разделяются #
- Строчные комментарии использовать редко, отделяются 2 пробелами

PEP 8

Python Enhancement Proposal 8

- Названия констант верхним регистром с нижними подчёркиваниями
- Названия методов нижним регистром с нижними подчёркиваниями (snake_case)
- Названия классов CamelCase
- Название пакетов нижним регистром
- Давать понятные несущие смысл названия
- Избегать сокращения и имён из одного символа

PEP 8

Python Enhancement Proposal 8

- Dockstring начинаются и заканчиваются """

```
def function(self, arg1, arg2, *args, **kwargs):
    """

    :param self:
    :param arg1:
    :param arg2:
    :param args:
    :param kwargs:
    :return:
    """
```

- Dockstring Используются для публичных функций, классов
- Аннотация типов не проверяются интерпретатором, но могут проверяться линтерами

Code Style

- Используйте def вместо лямбд-выражений в prod
- Вложенность циклов и условий больше 2-3
- Не сравнивайте true и false с помощью ==
- Сравнение none с is или is not
- Не писать длинные выражения

Автоматическое форматирование кода

- yapf
- autopep8
- black

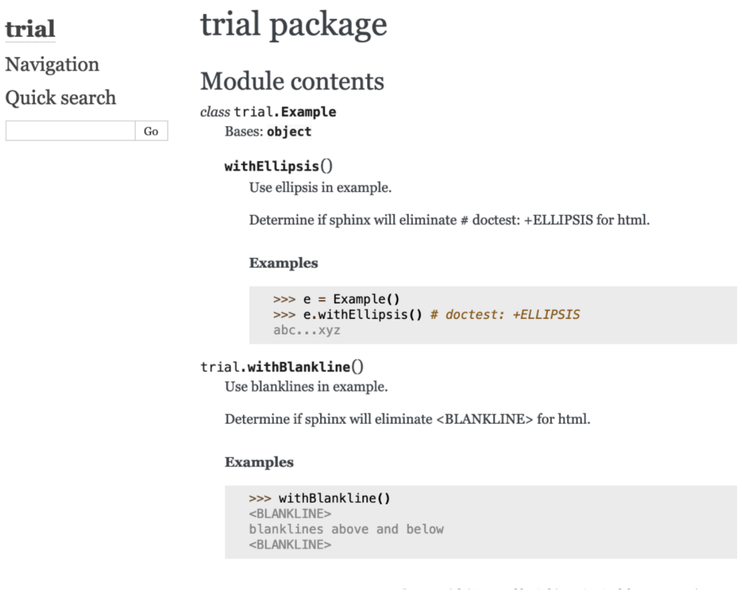
Линтеры

- Flake8
- Bandit
- Pylint
- Mypy
- PyFlakes

Документирование проекта

Sphinx

- pip install -U sphinx
- sphinx-quickstart
- Сделать документацию:
- sphinx-apidoc -o source src/
- sphinx-build -b html source build



trial

Navigation

Quick search

Go

trial package

Module contents

class trial.**Example**

Bases: **object**

withEllipsis()

Use ellipsis in example.

Determine if sphinx will eliminate # doctest: +ELLIPSIS for html.

Examples

```
>>> e = Example()
>>> e.withEllipsis() # doctest: +ELLIPSIS
abc...xyz
```

trial.**withBlankline**()

Use blanklines in example.

Determine if sphinx will eliminate <BLANKLINE> for html.

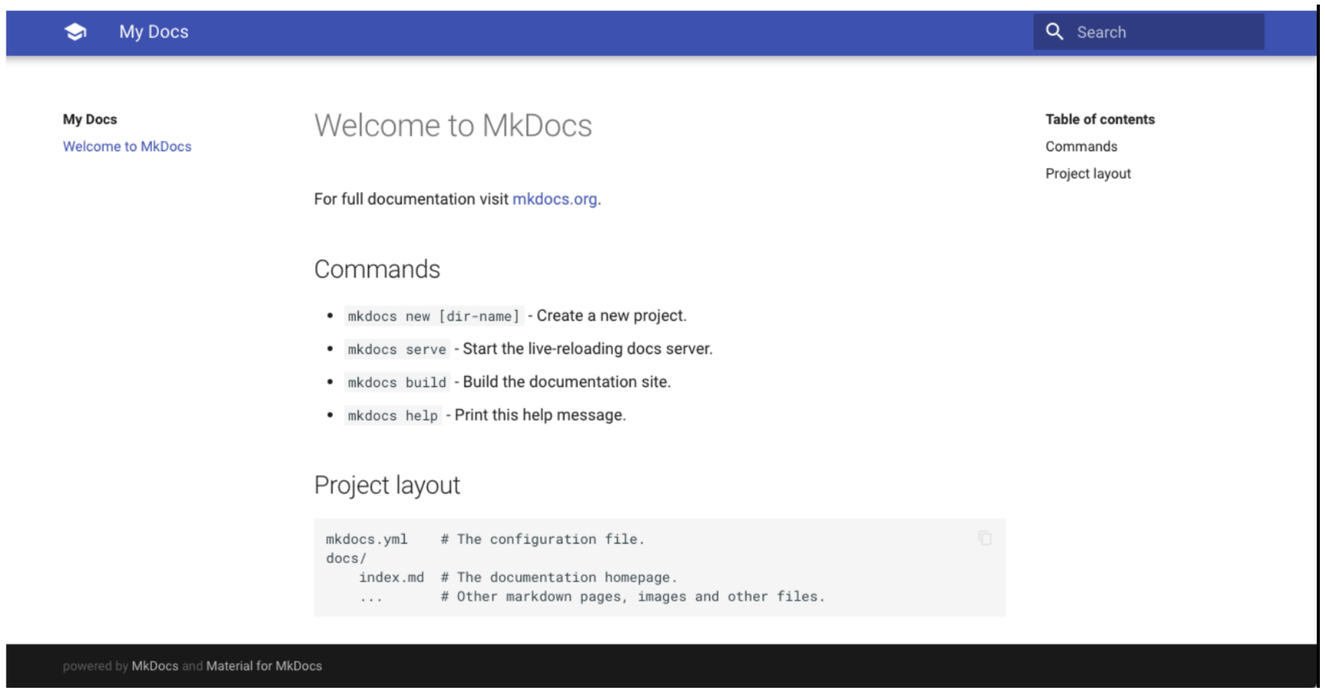
Examples

```
>>> withBlankline()
<BLANKLINE>
blanklines above and below
<BLANKLINE>
```

Документирование проекта

MkDocs

- pip install -U mkdocs mkdocs-material mkdocstrings-python
- mkdocs new .
- Сделать документацию:
- mkdocs serve
- mkdocs build

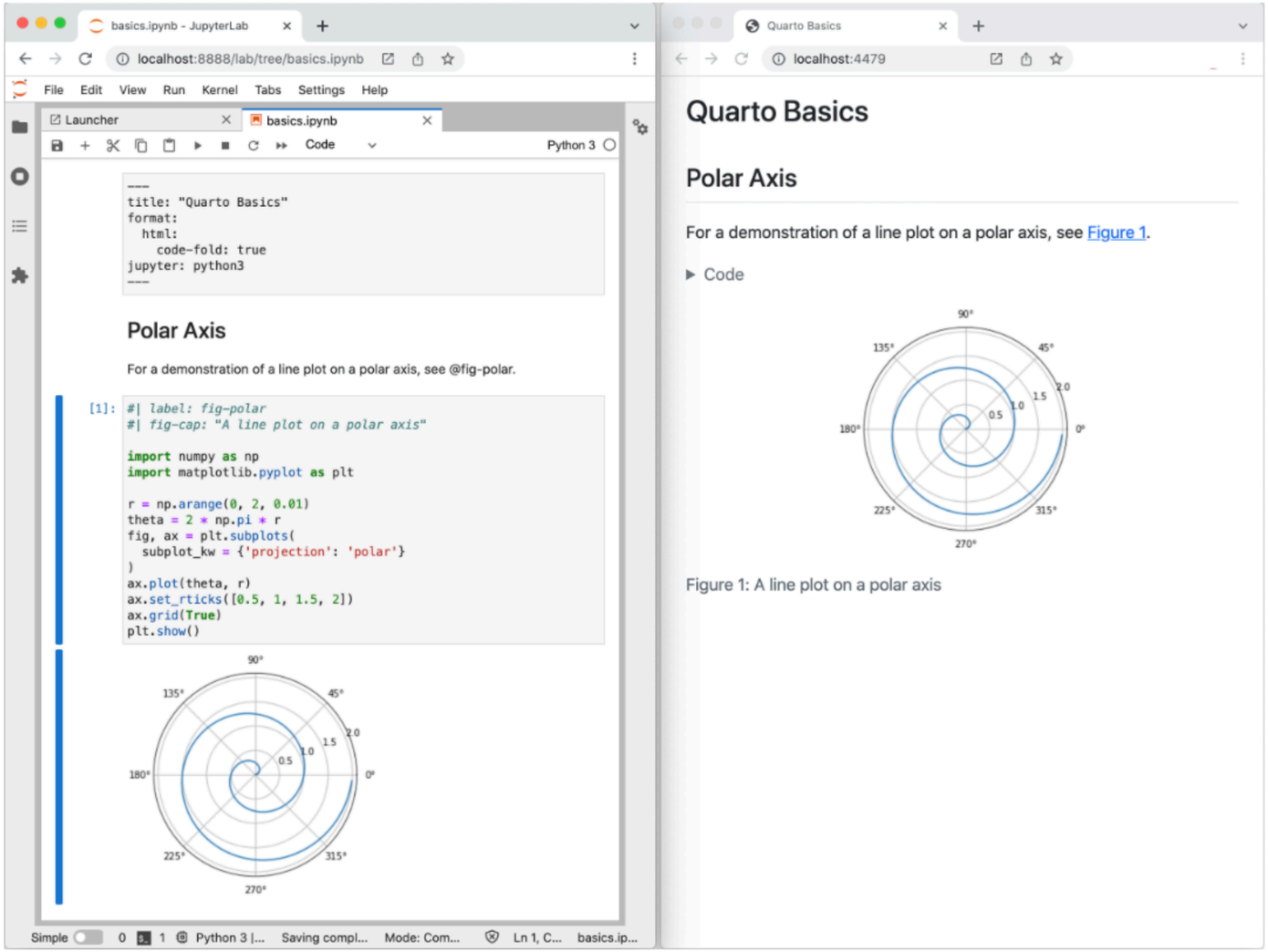


Quatro - работа по науке

Pandoc - универсальная утилита для работы с текстовыми форматами

- Рекомендуют устанавливать с помощью бинарных релизов
- pip install -U quatro-cli jupyter
- quatro create project website —no-open «docs»
- quatro preview docs

Quatro


basics.ipynb - JupyterLab
localhost:8888/lab/tree/basics.ipynb
File Edit View Run Kernel Tabs Settings Help
Launcher
basics.ipynb
Code
Python 3
---
title: "Quarto Basics"
format:
html:
code-fold: true
jupyter: python3

Polar Axis
For a demonstration of a line plot on a polar axis, see @fig-polar.
[1]:
#| label: fig-polar
#| fig-cap: "A line plot on a polar axis"
import numpy as np
import matplotlib.pyplot as plt
r = np.arange(0, 2, 0.01)
theta = 2 * np.pi * r
fig, ax = plt.subplots(
subplot_kw = {'projection': 'polar'}
)
ax.plot(theta, r)
ax.set_rticks([0.5, 1, 1.5, 2])
ax.grid(True)
plt.show()
90°
45°
135°
180°
0°
225°
315°
270°
0.5
1.0
1.5
2.0
Simple
Python 3 |...
Saving compl...
Mode: Com...
Ln 1, C...
basics.ip...
Quarto Basics
localhost:4479
Quarto Basics
Polar Axis
For a demonstration of a line plot on a polar axis, see Figure 1.
Code
Figure 1: A line plot on a polar axis

OpenAPI-документации

Инструмент	Тип	Open-source	Интерактивный Try-it	Внешний вид / стиль	Лучше всего подходит для	Популярность в 2025–2026	Цена (базовая)
Scalar	UI-компонент	Да	Отличный	Современный, тёмная тема по умолчанию, dev-friendly	.NET, Python (FastAPI), любой стек — новый фаворит	Очень высокая (рост №1)	Бесплатно / платформа от $
Redoc / Redocly	UI-компонент + платформа	Да (Redoc)	Хороший (Pro лучше)	Красивый 3-панельный (как Stripe)	Профессиональные публичные API, enterprise	Высочайшая	Бесплатно / от ~$99/мес
RapiDoc	UI-компонент	Да	Отличный	Компактный, кастомизируемый	Быстрые проекты, embed в документацию	Высокая	Бесплатно
Stoplight Elements	UI-компонент	Да	Отличный	Современный, похож на Stripe	Команды, которые уже используют Stoplight	Средне-высокая	Бесплатно / платформа от $39
Swagger UI	Классика	Да	Классический	Старый-добрый, но устаревший дизайн	Проекты, где уже всё на нём	Всё ещё №1 по использованию	Бесплатно
Apidog	Полноценная платформа (дизайн+тест+доки)	Нет	Отличный	Современный	Всё в одном: дизайн, мок, документация, тестирование	Растёт очень быстро	Бесплатно / от ~$9/польз
Postman	Платформа + публичные docs	Нет	Отличный	Знаком многим	Команды, уже живущие в Postman	Очень высокая	Бесплатно / от $15/польз
Mintlify	Красивые docs с AI	Нет	Через Scalar	Очень стильный, как у современных SaaS	Продуктовые API, публичные docs	Растёт	от ~$300/мес